st_04_principles

1. Abstractions, Examples

Please, visit https://www.studentenwerk-leipzig.de/mensen-cafeterien/speiseplan. Read the various offers for meals carefully. Develop categorization schemes for each meal offer in its collapsed state. Provide a structural description for each categorization scheme you identified. Please verify, that each meal offer is covered by one of your categorization schemes. If you describe your categories using classes do not use inheritance!

A “main dish” and „side dish“ consisting of a

- “header”
- zero, one or two “icon”s
- “name”
- 0 to three side dishes
- “compound price information”
- “optional hint/variants” or/and
- “optional expansion indicator”
- Indicator for Allergenes

2. Abstraction Levels, OMG Four Layers Standard Modeling Stack

1. Hopefully, you identified more than one category in the previous exercise. Now look at the vocabulary used for the descriptions. Try to categorize the vocabulary, name the categories, and explicitly map your vocabulary to these categories.
2. Now have a look at the slide describing the OMG four layers standard modeling stack. Which is the appropriate level for each of the following:
    a. the meal offers from https://www.studentenwerk-leipzig.de/mensen-cafeterien/speiseplan,
    b. your categorization schemes for the meal offers,
    c. and the categories for vocabulary identified in the previous subtask?

---

1. Three categories of vocabulary

First, there is the “category name”, namely “fish”, „smoothy“ and “vegetarian”…

Then there are property names, e.g., “name” and “compound price information”.

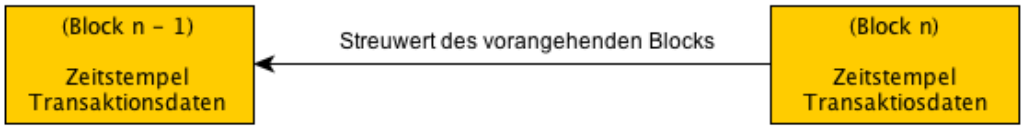
Eventually, we have a kind of specifier, “optional”, which tells, whether a property can be omitted, that is, it must not be assign a value.

2. The concrete meal offers are exemplars which belong to level M0. The categories resulting from the task before are models, hence, they belong to level M1. The result of the subtask above is a meta model, which belongs to level M2.

3. Structuring

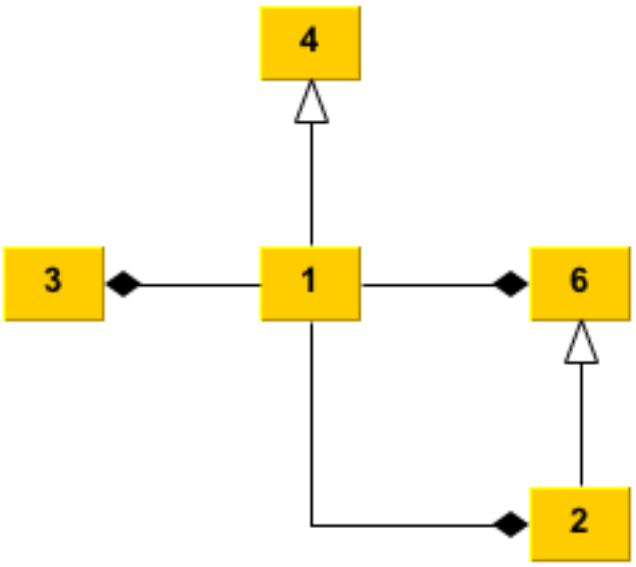
The first paragraph of the Wikipedia article “Blockchain” states verbatim “*Eine Blockchain … ist eine kontinuierlich erweiterbare Liste von Datensätzen, genannt „Blöcke“, welche mittels kryptographischer Verfahren miteinander verkettet sind. … Jeder Block enthält dabei*

typischerweise einen kryptographisch sicheren Hash (Streuwert) des vorhergehenden Blocks, … einen Zeitstempel und Transaktionsdaten.“ (https://de.wikipedia.org/wiki/Blockchain, accessed 10/23/2018). Please read this paragraph carefully, then tell which parts are connected by which relation. Refer to the preceding text exclusively! Draw a simple diagram.



4. Hierarchization

Look at the following UML class diagram.



1. Which hierarchies does it contain?
2. Now assume, that each class has a constructor which prints the class name. If a class A inherits from another class B, then the constructor of B is called before the constructor of A is executed. If a class C aggregates a class D, then the constructor of D is called before the constructor of C is executed. Furthermore, inheritance has a higher priority than aggregation with respect to constructor invocation. Given all that, write down the resulting cipher sequence when creating one exemplar of each class followed a newline character.

Inheritance & Aggregation

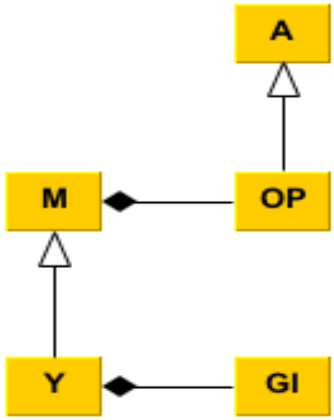
5. Hierarchization

Given are the following sentences which are produced by creating exemplars of different classes, each creation followed by a newline character, that is, the same mechanism as outlined in the preceding task:

A
AOP
AOPM
AOPMGIY
GI

Hint: It is possible that a class name has more than one character!

One possible solution is:



6. Modularization

Please, discuss whether a function can be appropriately classified as module! Can you conceive modules on a smaller scale than a function?

Yes, a function can definitively be a module.

It has a well-defined interface (input and output) via the function name and parameter list, and its inner parts should have a high cohesion.

7. Modularization

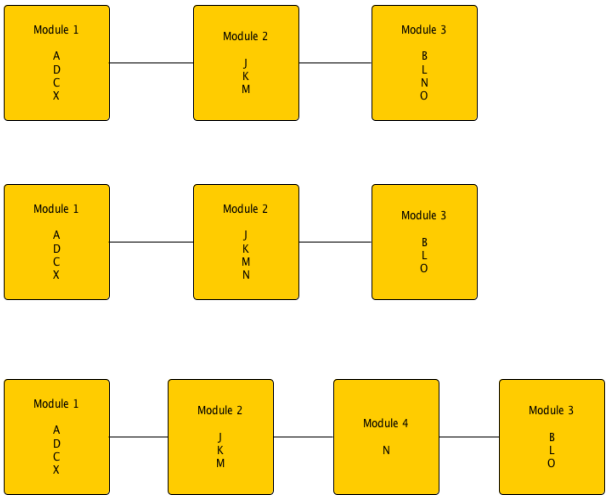
The following list shows small software entities and the connections among them:

1. A: A, D, X
2. B: B, J, L, N, O
3. C: A, C, D, N, X
4. D: D, J, K
5. J: D, M
6. K: D, M
7. L: B, L, N, O
8. M: J, K, N
9. N: B, C, L, O
10. O: B, L, N, O
11. X: A, C, D

“J: D, M” means, J and D are connected and J and M are connected. There is no information available, whether these connections are directed or not. Thus, you should not make any assumption concerning direction.

1. First, you should think about a plausibility check for the list above and apply it. Is the information in the list consistent? If not, try to repair it. Doing so, you may add items, but not remove them.
2. Next, think about another more explicit and more formal representation for the information contained in the list. Maybe you consider a table or a matrix.
3. Enter the information of the list into the new representation. Then analyze the entities and their connections. Afterwards you should try to transform the content of the new representation in a consistent way, in order to recognize groups or clusters. Now make a proposal how to modularize them. How many modules do result? Which entities are placed into which module? Are there connections among the modules you identified? If so, which? Motivate your decisions by applying the criteria cohesion and coupling for good modularization.

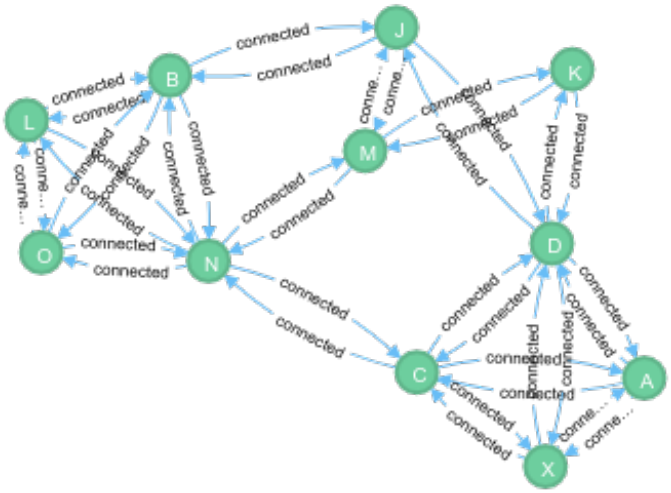
Module 1 with entities A, D, C, X
Module 2 with entities J, K, M
Module 3 with entities B, L, N, O
Module 1 and module 2 are linked, because of entity D. The interface of module 1 has a connection to D, the interface of module 2 has connections to J and K.
The connection between M and N is somewhat difficult. It links module 2 and module 3. Thus, we could move it either to module 2 or to module 3 or we decide, that it should be a module in its own. If so, it would be only element of module 4, and module 2 had to be connected with module 4 and module 4 had to be connected with module 3.



8. Rigor and Formality, Generalization

Revisit the task of modularization of entities and their connection and have a look at your solution and the solution presented by your exercise instructors. What is your guess, how high the effort is for finding a formal solution that can be applied to the complete class of similar problems?

	d	c	a	x	j	k	n	b	l	o	m
d	1	1	1	1	1	1	0	0	0	0	0
c	1	1	1	1	0	0	1	0	0	0	0
a	1	1	1	1	0	0	0	0	0	0	0
x	1	1	1	0	0	0	0	0	0	0	0
j	1	0	0	0	0	0	0	1	0	0	1
k	1	0	0	0	0	0	0	0	0	0	1
n	0	1	0	0	0	0	0	1	1	1	1
b	0	0	0	0	1	0	1	1	1	1	0
l	0	0	0	0	0	0	1	1	1	1	0
o	0	0	0	0	0	0	1	1	1	1	0
m	0	0	0	0	1	1	1	0	0	0	0



Is it in this case appropriate to devise such a formal solution to this problem? What is your opinion?

Decision depends on whether this is a recurring problem.
If so, the effort for finding such a solution may pay back.

Another reason for searching such a solution may be, that it is crucial to find a solution based on a formal procedure, because the solution is very important, e.g., life critical.

But both reasons do not apply to this task.

9. Information hiding

Have a closer look at classes in C++ and Java and access specifiers. Discuss which variant of information hiding is applied in both programming languages.

Both, C++ and Java, employ the liberal variant of information hiding. (public, private, protected)
C++ allows to tighten it a little more than Java, but only when using ordinary classes.
In this case it is possible to have a separate file declaring the prototypes (.h) and another file containing the complete definitions, a.k.a. implementations (.cpp).

12. All principles

Checkout risk_assessment.pl from the repository. This program was kindly developed by Andreas Barton, head of faculty service center. Please, assess this program according to the principles mentioned in the slides. For each principle you should state, whether it is applicable or not and why. If it is applicable, you should write down your result of assessing the program according to this principle.

Abstraction: Rather abstract - risk and verify conceal details behind simple calls.
Structure: Programme clearly structured into sections and tasks.
Hierarchisation: Present through the predicate calls, but is only faintly defined.
Modularisation: Building blocks are present, but no explicit module boundaries.
Information hiding: The global state remains unprotected.
Locality: Locality high, code already somewhat on one page
Verbalisation: The names are mostly understandable
Separation of concerns: Risk logic and dialogue are separate
Rigor and formality: Prolog makes it very formal, but the lack of input validation and order-dependent rules make verification difficult.
Anticipation of change: New risk factors require changes to several existing rules, but it is generally abstracted and structured.
Generality: The question management is reusable, whilst the risk logic is highly specialised.
Incrementality: Extensions are possible, but may affect the existing prioritisation.
Don’t repeat yourself: Identical rules and repeated question texts violate the DRY principle.

13. All principles

You are a member of a team that develops a large application using a programming language with strong static typing. The team feels, that development is not as smooth as possible. Almost each team member works on all parts of the software. Furthermore, the parameter lists of functions and methods need to be frequently adjusted to externally triggered changes. Thus, an external consultant had a closer look to your project. He argues, that locality is bad and that the frequent changes, which cannot be foreseen exactly, are not properly anticipated. As a remedy he proposes a fundamental design change. Functions and methods should be rewritten, so that they take only one parameter. This parameter should encapsulate all the previously individually passed parameters. The return value of a function or method can be a compound type too, if necessary. This way, the number of a parameters is kept fixed, i.e., one. If changes are necessary they are applied in one place only, the single parameter passed.
Now the proposal is discussed by the team. What is your opinion and how do you argue?

There is a design pattern called “parameter object”.

Statically typed language → Change from the passed parameters to a parameter object requires to adjust the respective type. Probably increases the number of parameter object types, which might cause additional maintenance effort.

Applying “anticipation of change” and “locality” to the project’s problems does not seem to be appropriate. It must always be checked carefully, whether a function is called with the appropriate parameter object. This requires frequent switching between callee and callers.

Another way: applying structuring, modularization, information hiding, and separation of concerns. First, identify those parts which are frequently affected by external changes. Build one module for those. Stable parts should be structured and partitioned into modules maximizing cohesion and minimizing coupling. Internals, especially those undergoing frequent changes, should be isolated consequently by information hiding. It could be also a good idea to apply “separation of concerns” and focus stable parts first. After finishing them the frequently changing, volatile parts should be analyzed more thoroughly.
Also maybe the external changes are the problem – so find out what triggers them and try to fix this.